

Conclusão de Projeto PHP - Todolist

O projeto implementa um sistema de lista de tarefas seguindo o padrão de arquitetura **MVC (Model-View-Controller)**, que separa as responsabilidades da aplicação em três camadas diferentes, facilitando a manutenção e o entendimento do código.

Camada de Modelo (Model) — As classes **User** e **Task** representam as entidades do sistema e são responsáveis exclusivamente pela comunicação com o banco de dados, usando **PDO** com queries preparadas para evitar SQL Injection. **User** cuida do cadastro e busca de usuários por e-mail, enquanto **Task** toma conta de toda a lógica das tarefas: criação, busca filtrada (por categoria, prioridade, texto e data), atualização, status e exclusão.

Camada de Visão (View) — Os arquivos dentro de **Views/** contêm apenas HTML com pequenos trechos de PHP para exibir dados, sem nenhuma regra de negócio. Isso mantém a interface desacoplada da lógica da aplicação.

Camada de Controle (Controller) — **AuthController** e **TaskController** determinam o fluxo entre Model e View: recebem dados das requisições HTTP **GET** e **POST**, aplicam validações, chamam os Models para manter e consultar dados, e decidem qual tela renderizar ou para onde redirecionar.

Padrões de projeto utilizados:

- **Front Controller**, representado pela classe **Router**: todas as requisições passam pelo **index.php**, que envia o tratamento a um único ponto de entrada (**Router::route()**), lá é interpretado o parâmetro **page** da URL e direciona para o Controller correto.
- **Factory Method**, implementado em **ModelFactory**: centraliza a criação de instâncias dos Models **User** e **Task** evitando que os Controllers conheçam diretamente as classes e facilitando a inclusão de novos Models no futuro (considerando que o projeto poderia vir a crescer).
- **Singleton**, presente em **Database**: faz com que tenha apenas uma conexão **PDO** ativa durante o uso da aplicação, essa conexão é reaproveitada por todos os Models.

Com isso a aplicação segue o seguinte fluxo: o usuário acessa uma URL → o **Router** identifica a página solicitada, o **Controller** processa a requisição usando a **ModelFactory** para obter o Model correto, o Model interage com o banco pela **Database** e o **Controller** carrega a **View** para exibir o resultado ao usuário.

Acredito que organizado desta forma o projeto segue as orientações passadas e fica sendo uma aplicação escalável onde novas funcionalidades podem ser adicionadas apenas criando novos Models, Controllers e Views, sem necessidade de alterar a lógica central de roteamento.

Deixei também 3 emails já cadastrados para facilitar os testes, os emails podem ser visualizados no banco e as senhas para cada um é 123456.